

SRE Lab Content

Application Performance Monitoring (APM): APM Setup and Backend Profiling using AWS

Performance Monitoring (APM) on AWS for backend profiling, monitoring response times, error rates, and throughput, and correlating performance with server metrics (CPU, memory).

Here's a **detailed step-by-step guide** to setting up **Application Performance Monitoring (APM)** on AWS, including **creating an EC2 instance using Terraform** and enabling necessary services.

◆ **Phase 1: Set Up Infrastructure Using Terraform**

We will use Terraform to **provision an EC2 instance** with the required IAM roles, security groups, and monitoring tools.

☑ **Step 1: Install Terraform & AWS CLI (if not installed)**

sudo apt update && sudo apt upgrade -y

Install Terraform

- Download and Install Terraform
<https://developer.hashicorp.com/terraform/install#darwinInstall>
- Verify installation
`terraform -version`

Install AWS CLI

- Download AWS CLI from

<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

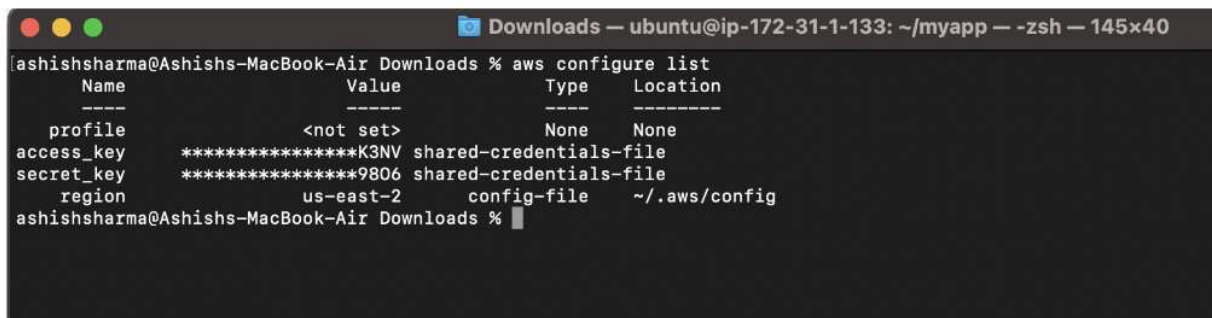
For deb/ubuntu download from below command.

```
curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
&& sudo installer -pkg AWSCLIV2.pkg -target /
```

```
aws --version
```

- Configure AWS credentials using below command.

```
aws configure
```



```
ashishsharma@Ashishs-MacBook-Air Downloads % aws configure list
```

Name	Value	Type	Location
profile	<not set>	None	None
access_key	*****K3NV	shared-credentials-file	
secret_key	*****9806	shared-credentials-file	
region	us-east-2	config-file	~/.aws/config

```
ashishsharma@Ashishs-MacBook-Air Downloads %
```

Provide:

- **AWS Access Key**
- **AWS Secret Key**
- **Default Region (e.g., us-east-1)**

Step 2: Create Terraform Configuration for EC2

Create a directory and files:

```
mkdir terraform-apm
cd terraform-apm
touch main.tf variables.tf outputs.tf
```

`sudo vi main.tf` (Terraform Configuration)

Add the following Terraform configuration:

```
provider "aws" {
  region = "us-east-2" # Change this to your region
}

resource "aws_instance" "apm_ec2" {
  ami           = "ami-0c55b159cbfaffe1f0" # Change based on your region
  instance_type = "t2.micro"
  key_name      = "your-key-name" # Ensure you have an SSH key
  iam_instance_profile = aws_iam_instance_profile.apm_profile.name
  security_groups = [aws_security_group.apm_sg.name]

  tags = {
    Name = "APM-EC2-Instance"
  }
}

resource "aws_security_group" "apm_sg" {
  name        = "apm-security-group"
  description = "Allow inbound access"

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 80
  }
}
```

```

    to_port    = 80
    protocol   = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 3000
    to_port   = 3000
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_iam_role" "apm_role" {
  name = "APMRole"

  assume_role_policy = <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "ec2.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}

```

EOF

}

```
resource "aws_iam_role_policy_attachment" "cloudwatch_attach" {
  role      = aws_iam_role.apm_role.name
  policy_arn = "arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy"
}
```

```
resource "aws_iam_instance_profile" "apm_profile" {
  name = "APMInstanceProfile"
  role = aws_iam_role.apm_role.name
}
```

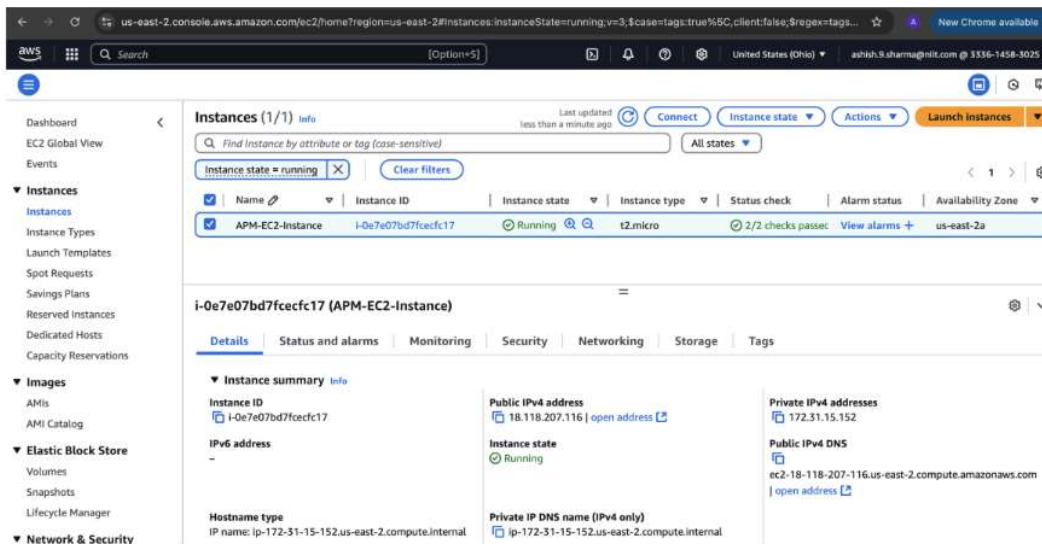
```
output "ec2_public_ip" {
  value = aws_instance.apm_ec2.public_ip
}
```

✓ Step 3: Deploy EC2 Using Terraform

1. **Initialize Terraform** `sh terraform init`
2. **Plan the Deployment** `terraform plan`
3. **Apply the Configuration**
`terraform apply -auto-approve`

Get the Public IP

```
terraform output ec2_public_ip
```

✓ Step 4: Install & Configure APM Tools

Now that we have an EC2 instance (check in aws console > instance) , we now install **AWS X-Ray**, **CloudWatch Agent**, and deploy a sample backend application (Node.js).

Login into instance : SSH into EC2 Instance

```
chmod 400 "<pem file>"
ssh -i your-key.pem ec2-user@<EC2_PUBLIC_IP>
```

✓ Step 5: Download and Install AWS X-Ray Daemon

Run the following commands to install the X-Ray daemon. To install the **AWS X-Ray Daemon** on an Ubuntu system, follow these steps:

Update Your System

Before installing any package, update your package lists:

```
sudo apt update && sudo apt upgrade -y
```

Download and Install AWS X-Ray Daemon

Run the following commands to install the X-Ray daemon:

```
wget https://s3.amazonaws.com/aws-xray-assets.us-east-1/xray-daemon/aws-xray-daemon-3.x.deb
```

Install the package:

```
sudo dpkg -i aws-xray-daemon-3.x.deb
```

If any dependencies are missing, install them:

```
sudo apt --fix-broken install -y
```

The `/etc/default/xray` file is the **configuration file** for the AWS X-Ray Daemon. It allows you to **set environment variables** to control how the daemon runs.

```
vi /etc/default/xray
```

```
.....
```

```
# AWS X-Ray Daemon Configuration
```

```
# The AWS Region where X-Ray should send traces
```

```
XRAY_REGION=us-east-1 # Change this to your AWS region
```

```
# The log level for X-Ray Daemon (Valid values: dev, info, warn, error)
```

```
XRAY_LOG_LEVEL=info
```

```
# The address on which the daemon listens for incoming data
```

```
XRAY_DAEMON_LISTEN_ADDRESS=127.0.0.1:2000
```

```
# Enable proxy mode (if needed)
```

```
XRAY_DAEMON_PROXY_MODE=false
```

```
# Additional configuration options (leave as default unless needed)
XRAY_DAEMON_NO_TRACE_ID=false
```

Explanation of Each Setting:

1. **XRAY_REGION** – The AWS region where traces should be sent.
2. **XRAY_LOG_LEVEL** – The verbosity of logs (`info`, `warn`, `error`).
3. **XRAY_DAEMON_LISTEN_ADDRESS** – The IP and port where the X-Ray daemon listens for trace data (default: `127.0.0.1:2000`).
4. **XRAY_DAEMON_PROXY_MODE** – Set to `true` if running behind a proxy.
5. **XRAY_DAEMON_NO_TRACE_ID** – Set to `true` if you want the daemon to process traces without a trace ID.

Start the X-Ray Daemon

Once installed, start the X-Ray daemon using:

```
sudo systemctl start xray
```

Enable it to start on boot:

```
sudo systemctl enable xray
```

Check if it's running:

```
systemctl status xray
```

Expected output:

```
● xray.service - AWS X-Ray Daemon
   Loaded: loaded (/etc/systemd/system/xray.service;
  enabled; vendor preset: enabled)
   Active: active (running) since ...
```

.....

Check X-Ray Daemon Logs

If traces are not appearing, check the daemon logs:

```
sudo journalctl -u xray --no-pager | tail -20
```

..... *

✓ Step 6: Install & Configure CloudWatch Agent

1. Install CloudWatch Agent

```
wget https://s3.amazonaws.com/amazoncloudwatch-agent/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb
```

Install the package:

```
sudo dpkg -i amazon-cloudwatch-agent.deb
```

If dependencies are missing, fix them using:

```
sudo apt --fix-broken install -y
```

Configure CloudWatch Agent:

```
sudo vi /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json
```

Add the following configuration:

```
{
  "agent": {
    "metrics_collection_interval": 60
  },
}
```

```
"metrics": {
  "metrics_collected": {
    "cpu": {
      "measurement": ["cpu_usage_idle", "cpu_usage_user"],
      "metrics_collection_interval": 60
    },
    "mem": {
      "measurement": ["mem_used_percent"],
      "metrics_collection_interval": 60
    }
  }
}
```

Start and Enable CloudWatch Agent:

```
sudo systemctl start amazon-cloudwatch-agent
sudo systemctl enable amazon-cloudwatch-agent
sudo systemctl status amazon-cloudwatch-agent
```

✓ Step 7: Deploy Sample Backend App

Install Node.js:

```
sudo apt install npm
```

Create a directory:

```
mkdir myapp && cd myapp
```

Initialize Node.js App:

```
npm init -y
```

install Express and AWS X-Ray:

```
npm install express aws-xray-sdk
```

Create `server.js`:

```
vi server.js
```

Add the following config:

```
const AWSXRay = require('aws-xray-sdk');
const express = require('express');
const app = express();

AWSXRay.captureHTTPsGlobal(require('http'));
AWSXRay.captureHTTPsGlobal(require('https'));

app.use(AWSXRay.express.openSegment('MyApp'));

app.get('/', (req, res) => {
  res.send('APM Monitoring Active!');
});

app.use(AWSXRay.express.closeSegment());

app.listen(3000, () => console.log('Server running on port 3000'));
```

Run the App:

```
node server.js
```

```

Downloads — ubuntu@ip-172-31-15-152: ~/myapp — ssh -i sre_demo.pem ubuntu@ec2-18-118-207-116.us-east-2.compute.amazonaws.com — 1...
ubuntu@ip-172-31-15-152:~$ cd myapp/
ubuntu@ip-172-31-15-152:~/myapp$ ls
node_modules  nohup.out  package-lock.json  package.json  server.js
ubuntu@ip-172-31-15-152:~/myapp$ cat server.js
const AWSXRay = require('aws-xray-sdk');
const express = require('express');
const app = express();

AWSXRay.captureHTTPGlobal(require('http'));
AWSXRay.captureHTTPGlobal(require('https'));

app.use(AWSXRay.express.openSegment('MyApp'));

app.get('/', (req, res) => {
  res.send('APM Monitoring Active!');
});

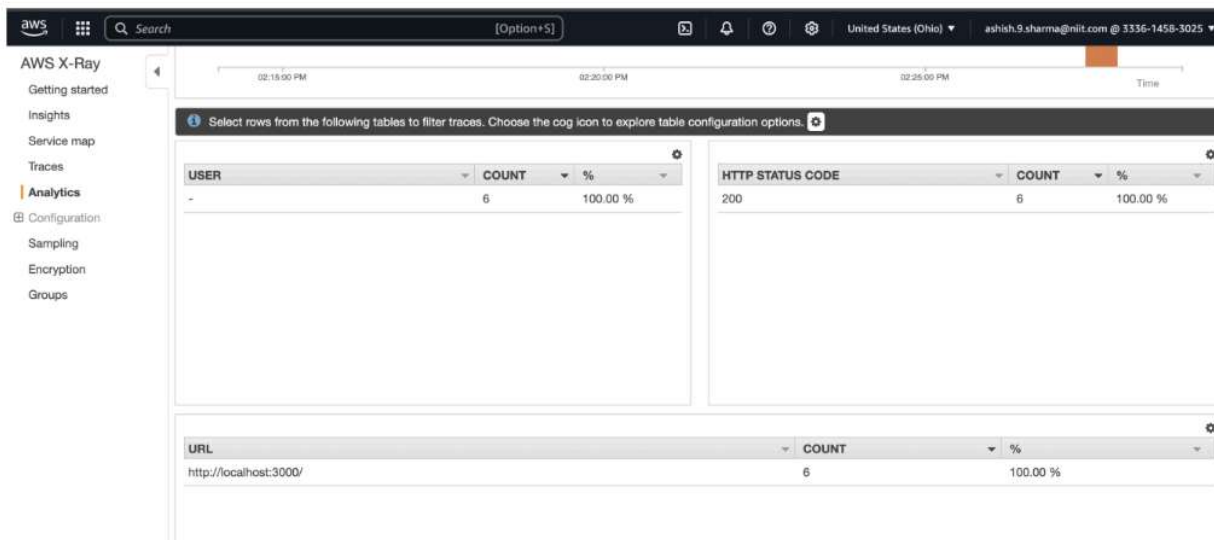
app.use(AWSXRay.express.closeSegment());

app.listen(3000, () => console.log('Server running on port 3000'));
ubuntu@ip-172-31-15-152:~/myapp$ █

```

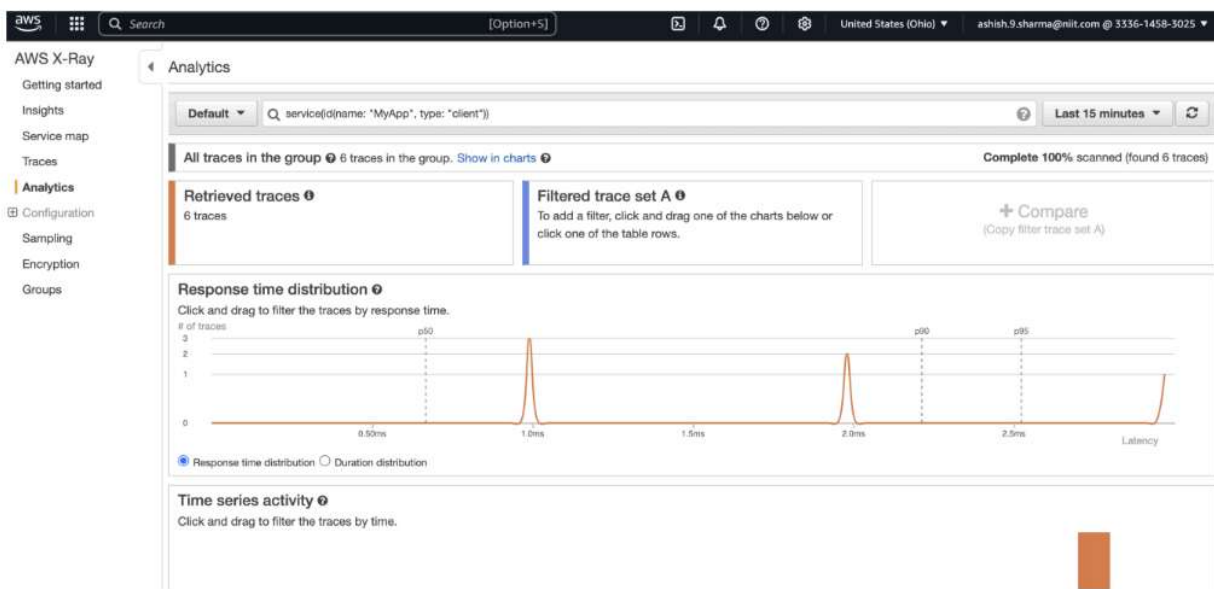
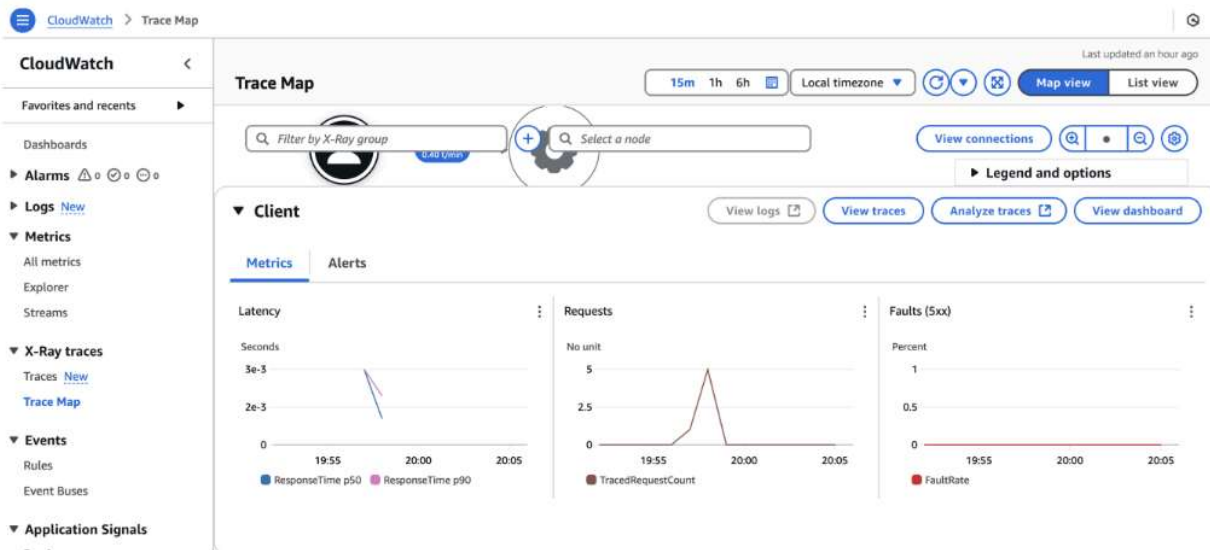
.....

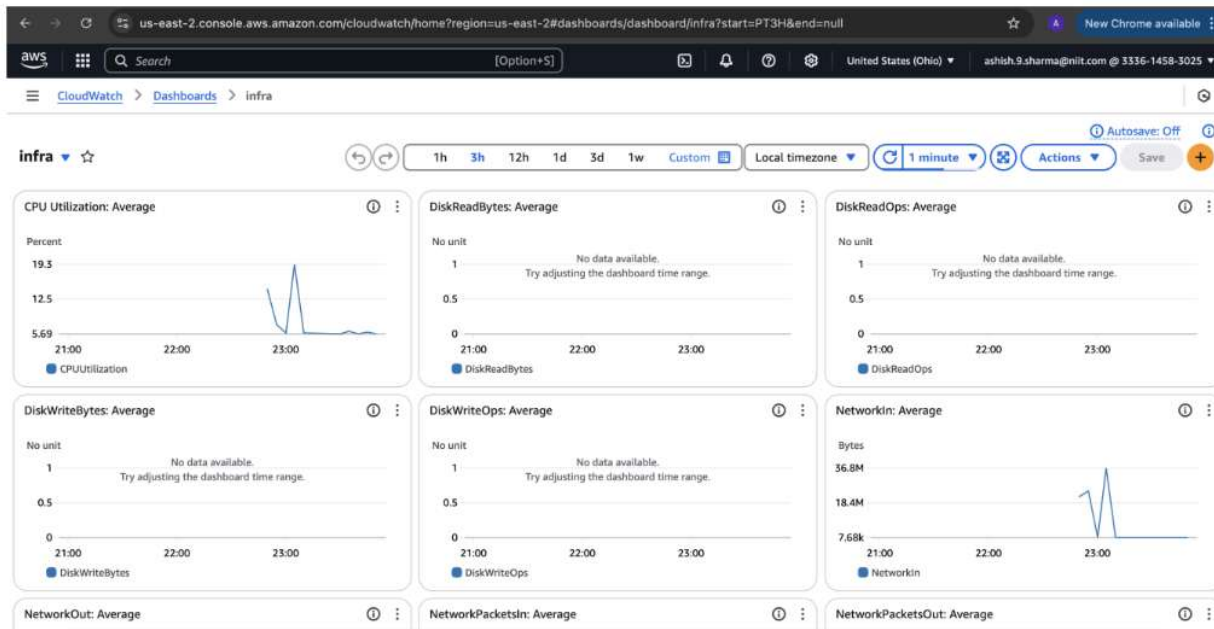
Hit : `curl http://localhost:3000` to verify the trace on x-ray service in aws for your application.



Step 8: Verify in AWS Console

1. Go to AWS X-Ray → Check Traces
2. Go to CloudWatch → Metrics
3. Create Dashboards & Alerts for High Latency or Errors





Concept behind above Implementation:

- ✓ Enable AWS X-Ray for APM tracing requests.
- ✓ Install and configure CloudWatch Agent for application logs & server metrics.
- ✓ Use CloudWatch Application Insights for backend performance monitoring.
- ✓ Correlate APM data with CPU & memory to find performance bottlenecks.
- ✓ Set up CloudWatch Alarms for real-time issue detection.

This setup provides **end-to-end Application Performance Monitoring (APM) on AWS** with **tracing, logging, and alerting**.

1. AWS X-Ray for APM (Backend Profiling)

AWS X-Ray helps you monitor and analyze the performance of your backend applications. It allows you to trace requests, monitor response times, detect bottlenecks, and identify issues in your application.

Step : Instrument your Application to Send Traces

- For AWS SDK-based applications (Node.js, Java, Python, etc.), integrate the AWS X-Ray SDK.
- Example for **Node.js**: `npm install aws-xray-sdk`

Step : View Traces in the AWS Console

1. Go to **AWS Console** → **X-Ray**
2. Click on **Service Map** to see the interactions between components.
3. You can filter traces by latency, error rates, etc.

2. Monitor Response Times, Error Rates, and Throughput (CloudWatch)

CloudWatch is another AWS service that helps monitor metrics such as response times, error rates, and throughput.

3. Correlating Performance with Server Metrics (CPU, Memory)

CloudWatch allows you to correlate **application performance** (from X-Ray and custom CloudWatch metrics) with **server metrics** (like CPU and memory usage).

Use CloudWatch Dashboards

Create a **CloudWatch Dashboard** to monitor both application performance metrics (like response times and error rates) and server-level

metrics (like CPU, memory, and network).

Go to CloudWatch → Dashboards → Create dashboard. Add widgets for metrics from both **CloudWatch Metrics** and **X-Ray**.

You can correlate the metrics by adding both **server metrics** (CPU, memory) and **application-level metrics** (response time, throughput).

Set Alarms for Metrics

You can create CloudWatch **Alarms** to trigger when specific thresholds are crossed. For example:

- High response time
- High error rate
- High CPU usage

For instance, you can set an alarm for when the **CPU utilization** goes above 80% or when the **response time** exceeds a certain threshold.

4. Visualize and Analyze Data

- **X-Ray Service Map:** To visualize the interdependencies between your services.
- **CloudWatch Dashboards:** To create custom dashboards combining backend application metrics and system-level metrics like CPU and memory usage.
- **CloudWatch Logs Insights:** For more advanced querying of logs related to your application's performance.

Summary of Key Steps:

1. **Install AWS X-Ray Daemon and instrument your application.**
2. **Monitor application metrics** like response time, error rate, and throughput using **CloudWatch** and **X-Ray**.
3. **Install CloudWatch Agent** for system metrics like CPU and memory.
4. **Correlate application performance with server metrics** using **CloudWatch Dashboards**.

- ✓ **Terraform:** Created EC2 with IAM roles & security groups
- ✓ **X-Ray:** Enabled distributed tracing
- ✓ **CloudWatch Agent:** Configured for CPU, memory, and logs
- ✓ **Backend App:** Deployed with APM monitoring
- ✓ **AWS Console:** Verified metrics & logs

.....

KB: To install the AWS CLI in aws console on Ubuntu, follow these steps:

Step 1: Update the Package Repository

Open your terminal and run the following command to ensure your package repository is up to date:

```
sudo apt update
```

Step 2: Install Required Dependencies

Install the necessary dependencies:

```
sudo apt install curl unzip -y
```

Step 3: Download the AWS CLI Installation Script

Use `curl` to download the AWS CLI installation script:

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
```


Step 4: Unzip the Installation Package

Unzip the downloaded file:

```
unzip awscliv2.zip
```

Step 5: Install AWS CLI

Run the installation command:

```
sudo ./aws/install
```

Step 6: Verify Installation

Check that AWS CLI is installed successfully by running:

```
aws --version
```

.....

Extra useful Command: nohup node server.js &

env | grep AWS

aws configure

aws configure list

Vi ~/.aws/credentials

Vi ~/.aws/config

Thank you